

使用 Kotlin 在 Android 中接收位置資訊更新

來源：<https://codelabs.developers.google.com/codelabs/while-in-use-location?hl=zh-tw>

步驟數：8

瞭解如何接收 Android 裝置的位置資訊 (包括 Android 11)。

目錄

1. [1. 事前準備](#)
2. [2. 開始使用](#)
3. [3. 新增位置資訊](#)
4. [4. 新增地點功能](#)
5. [5. 支援 Android 10](#)
6. [6. 支援 Android 11](#)
7. [7. Android 適用的位置資訊策略](#)
8. [8. 恭喜](#)

步驟 1 · 約 5 分鐘

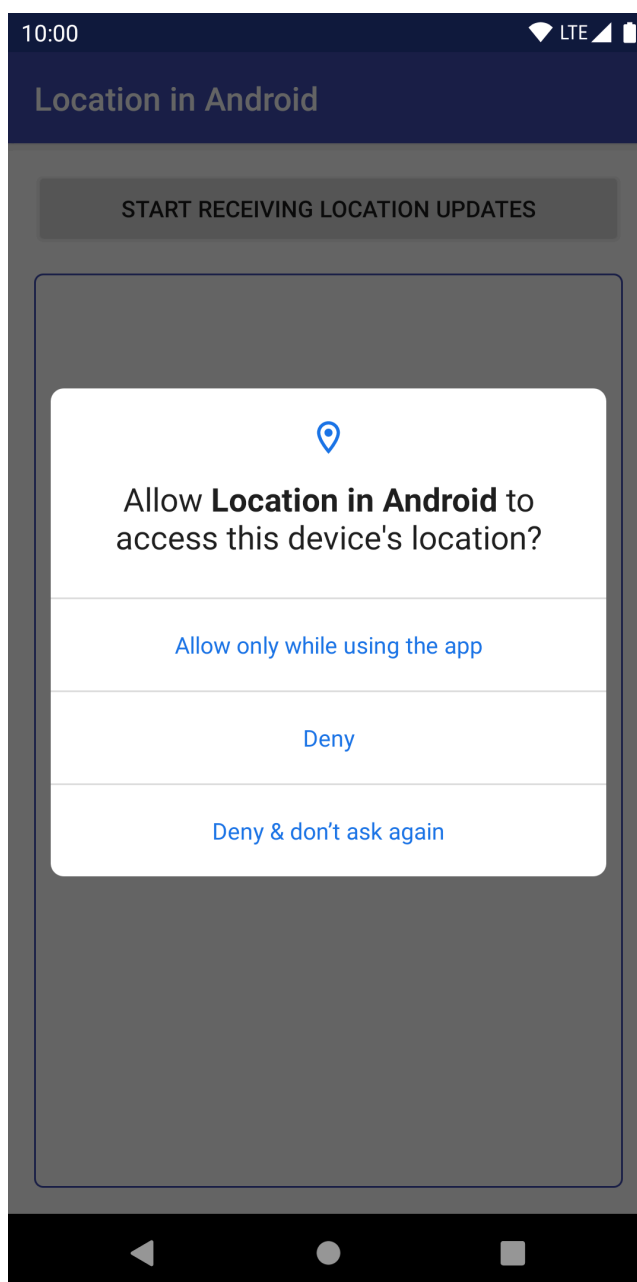
1. 事前準備

Android 10 和 11 可讓使用者進一步控管應用程式存取裝置位置資訊的權限。

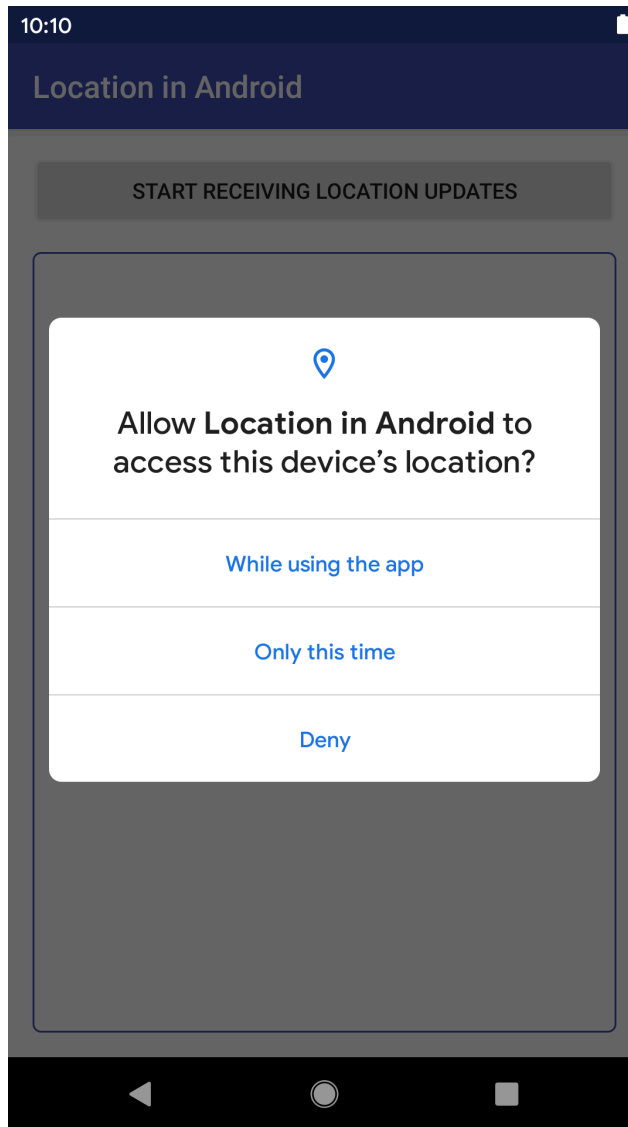
在 Android 11 上執行的應用程式要求位置存取權時，使用者有四個選項：

- 一律允許
- 僅在使用該應用程式時允許 (Android 10)
- 僅限一次 (Android 11)
- 拒絕

Android 10



Android 11



在本程式碼研究室中，您將瞭解如何接收位置資訊更新，以及如何在任何 Android 版本 (特別是 Android 10 和 11) 上支援位置資訊。完成本程式碼研究室後，您應該就能建立應用程式，並遵循目前擷取位置資訊更新的最佳做法。

必要條件

- 熟悉 Android 開發
- 熟悉[活動](#)、[服務](#)和[權限](#)

執行步驟

- 遵循 Android 位置資訊的最佳做法。
- 處理前景位置資訊存取權 (使用者要求應用程式在處於使用中狀態時存取裝置位置資訊)。
- 修改現有應用程式，加入訂閱及取消訂閱位置資訊的程式碼，以便支援要求位置資訊存取權。
- 在應用程式中新增邏輯，以便在前景位置資訊或使用中存取位置資訊，藉此支援 Android 10 和 11。

軟硬體需求

- Android Studio 3.4 以上版本，用於執行程式碼
 - 搭載 Android 10 和 11 開發人員預覽版的裝置/模擬器
-

步驟 2 · 約 7 分鐘

2. 開始使用

複製範例專案存放區

為了方便您盡快上手，您可以從這個新手範例專案開始建構。如果您已安裝 Git，只要執行下列指令即可：

```
git clone https://github.com/android/codelab-while-in-use-location
```

歡迎直接前往 [GitHub 頁面](#)。

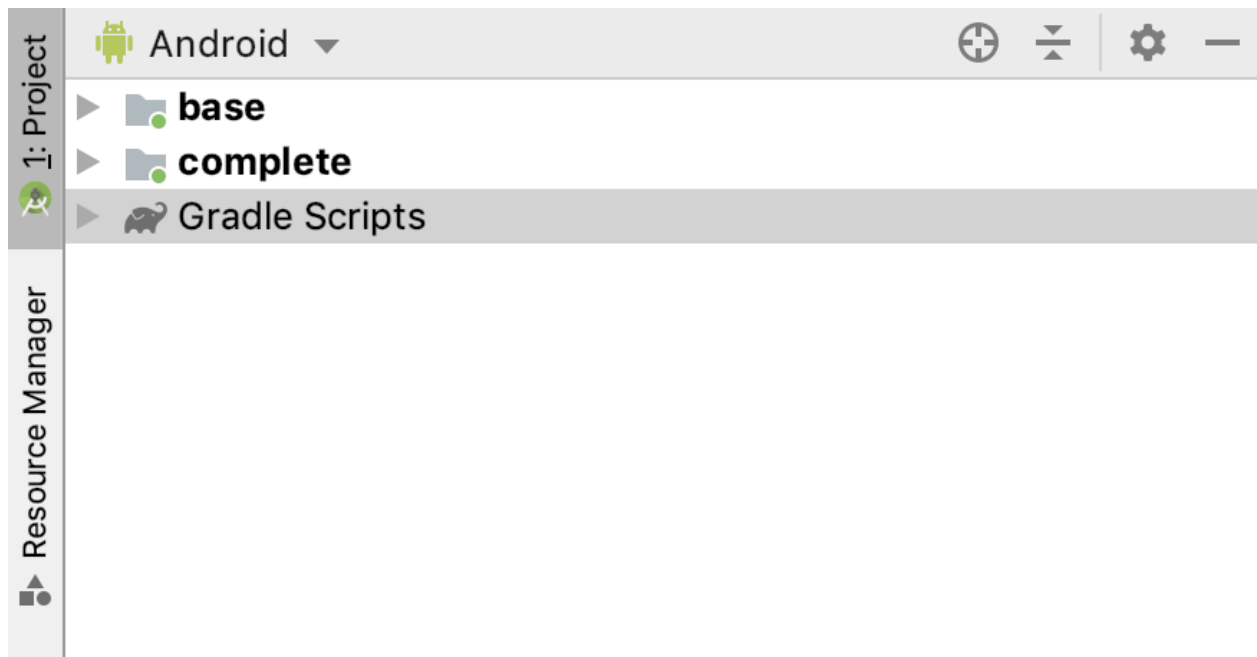
如果您沒有 Git，可以將專案下載為 ZIP 檔案：

匯入專案

開啟 Android Studio，然後在歡迎畫面中選取「Open an existing Android Studio project」，然後開啟專案目錄。

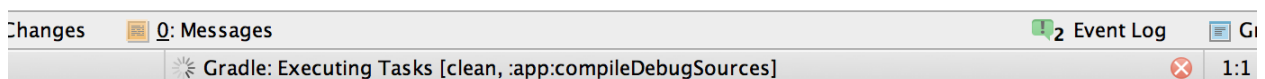
專案載入後，系統也可能會顯示快訊，通知您 Git 不會追蹤所有本機變更。你可以點選「忽略」。(系統不會將任何變更推送回 Git 存放區。)

如果您在「Android」檢視畫面中，專案視窗的左上角會顯示如下圖所示的項目 **Android**(在「Project」檢視畫面中，您必須展開專案才能看到這些項目)。



共有兩個資料夾 (`base` 和 `complete`)。每個資料夾都稱為「模組」。

請注意，第一次開啟專案時，Android Studio 可能需要花幾秒的時間在背景中編譯專案。在此期間，Android Studio 底部的狀態列會顯示下列訊息：



請等待 Android Studio 完成專案編列索引和建構作業，再變更程式碼。這使 Android Studio 提取所有必要的元件。

如果畫面顯示「Reload for language changes to take effect?」的提示或類似內容，請選取「Yes」。

瞭解範例專案

您已完成設定，可以開始在應用程式中要求位置資訊。請使用 `base` 模組做為起點。在每個步驟中，將程式碼新增至 `base` 模組。完成本程式碼研究室後，`base` 模組中的程式碼應與 `complete` 模組的內容相符。`complete` 模組可用於檢查你的作品，在發生問題時也可以參考。

主要元件包括：

- `MainActivity` — 使用者介面，可讓使用者允許應用程式存取裝置位置資訊
- `LocationService`：訂閱及取消訂閱位置資訊變更的服務，並在使用者離開應用程式活動時，升級為前景服務 (附帶通知)。在此新增位置代碼。
- `Util`：為 `Location` 類別新增擴充功能函式，並將位置資訊儲存到 `SharedPreferences` (簡化資料層)。

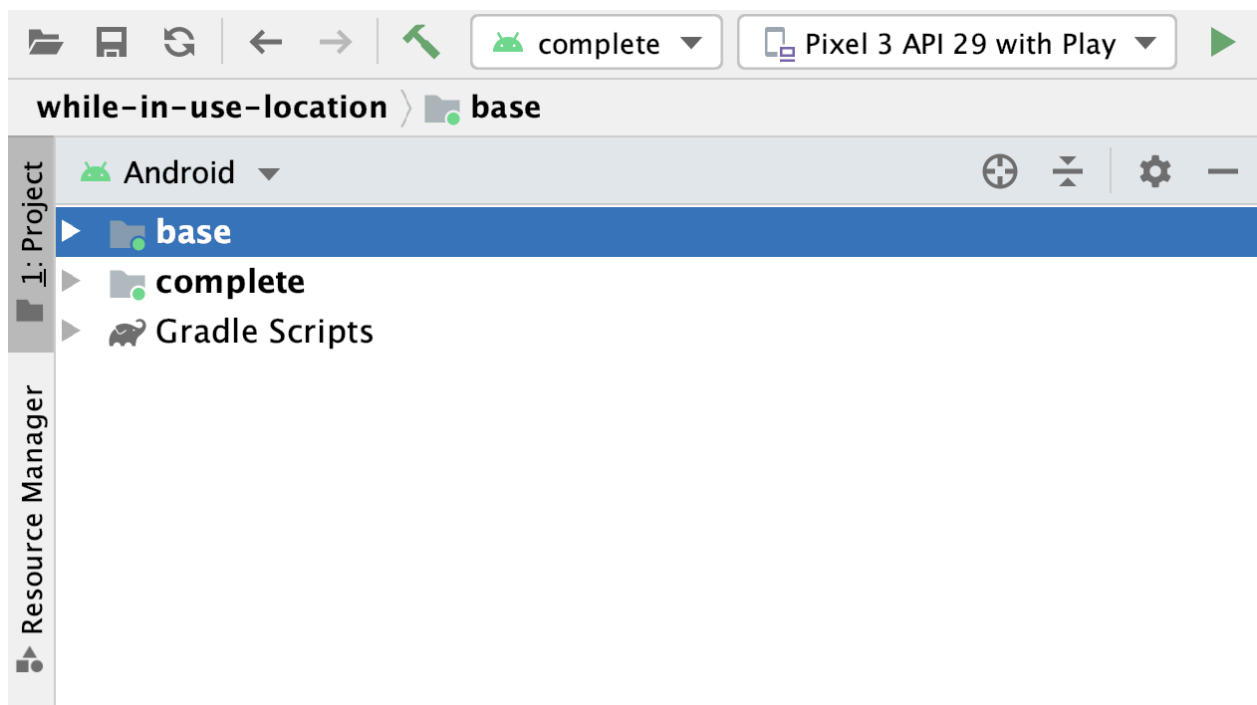
模擬器設定

若要瞭解如何設定 Android 模擬器，請參閱「[在模擬器上執行](#)」。

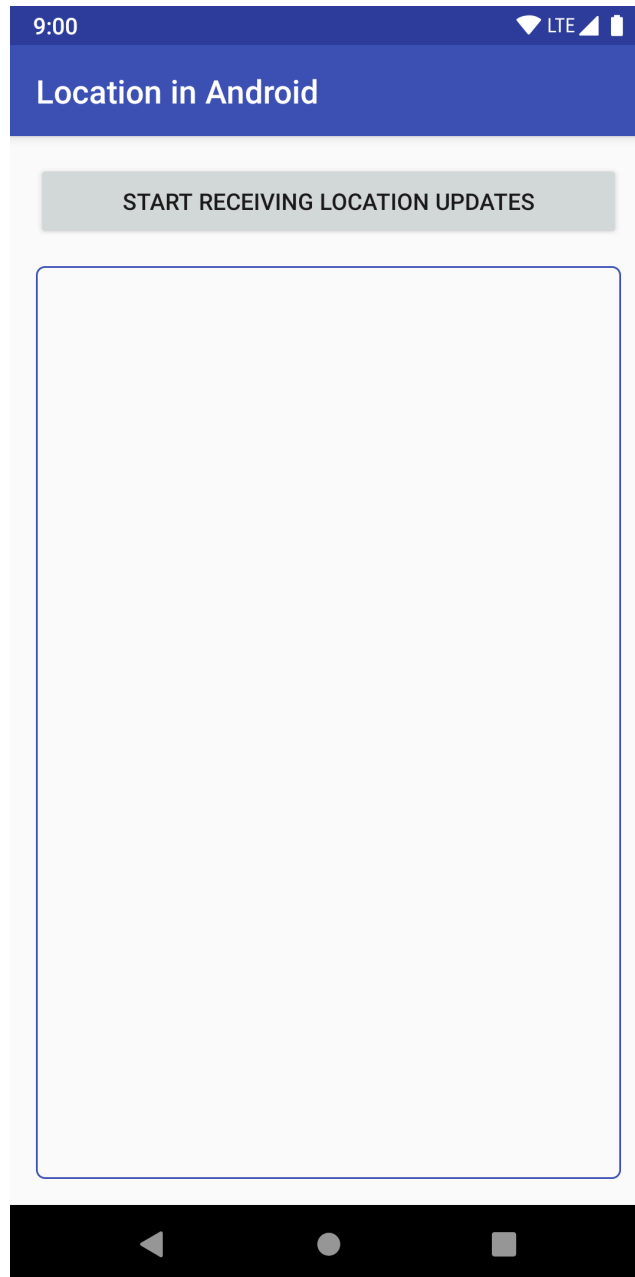
執行範例專案

執行應用程式。

1. 將 Android 裝置連接到電腦，或啟動模擬器。(確認裝置搭載 Android 10 以上版本)。
2. 在工具列中，從下拉式選取器中選取 `base` 設定，然後按一下「Run」(執行)：



3. 請注意，裝置上會顯示下列應用程式：



你可能會發現輸出畫面中未顯示任何位置資訊。這是因為您尚未加入地點代碼。

3. 新增位置資訊

概念

本程式碼研究室的重點是說明如何接收位置更新，並最終支援 Android 10 和 Android 11。

不過，在開始編寫程式碼之前，建議先複習基本概念。

位置存取權類型

您可能還記得，在本程式碼研究室的開頭，我們介紹了四種不同的位置資訊存取選項。各項狀態的說明如下：

- **僅在使用該應用程式時允許**
 - 這是適用於大多數應用程式的建議選項。這項選項又稱為「使用時」或「僅限前景」存取權，是在 Android 10 中新增，可讓開發人員僅在使用者主動使用應用程式時，擷取位置資訊。如果符合下列任一條件，應用程式就會視為處於活動狀態：
 - 活動會顯示在畫面上。
 - 前景服務正在執行，並顯示持續性通知。
 - **僅限一次**
 - 這項權限是在 Android 11 中新增，與「僅在使用該應用程式時允許」相同，但僅限一段時間。詳情請參閱「[單次權限](#)」。
 - **拒絕**
 - 這個選項會禁止存取位置資訊。
 - **一律允許**
 - 這個選項可讓應用程式隨時存取位置資訊，但[必須取得額外權限](#)，才能在 Android 10 以上版本中使用。此外，您也必須確保有正當用途，並遵守[位置資訊政策](#)。本程式碼研究室不會介紹這個選項，因為這是較少見的用途。不過，如果您有正當用途，並想瞭解如何正確處理「一律允許」位置資訊存取權 (包括在背景存取位置資訊)，請參閱[LocationUpdatesBackgroundKotlin 範例](#)。

服務、前景服務和繫結

如要完整支援「僅在使用該應用程式時允許」位置資訊更新，您必須考量使用者何時會離開應用程式。如要在這種情況下繼續接收更新，您需要建立前景 `Service`，並將其與 `Notification` 建立關聯。

此外，如果您想在應用程式顯示時，以及使用者離開應用程式時，使用相同的 `Service` 要求位置資訊更新，則需要將該 `Service` 繫結/取消繫結至 UI 元素。

由於本程式碼研究室只著重於取得位置資訊更新，因此您可以在 `ForegroundOnlyLocationService.kt` 類別中找到所需的所有程式碼。您可以瀏覽該類別和 `MainActivity.kt`，瞭解兩者如何搭配運作。

詳情請參閱「[服務總覽](#)」和「[繫結服務總覽](#)」。

權限

如要從 `NETWORK_PROVIDER` 或 `GPS_PROVIDER` 接收位置資訊更新，您必須在 Android 資訊清單檔案中分別宣告 `ACCESS_COARSE_LOCATION` 或 `ACCESS_FINE_LOCATION` 權限，以要求使用者授予權限。如果沒有這些權限，應用程式就無法在執行階段要求存取位置資訊。

在搭載 Android 10 以上版本的裝置上使用應用程式時，這些權限涵蓋「僅限這次」和「僅在使用該應用程式時允許」的情況。

位置

應用程式可以透過 `com.google.android.gms.location` 套件中的類別，存取一組支援的定位服務。

查看主要類別：

- `FusedLocationProviderClient`
- 這是位置資訊架構的中心元件，建立完成後，您可以使用這個用戶端要求位置更新，並取得最後已知位置。
- `LocationRequest`
- 這是資料物件，內含要求的服務品質參數 (更新間隔、優先順序和準確度)。要求更新位置資訊時，系統會將這個值傳遞至 `FusedLocationProviderClient`。
- `LocationCallback`
- 這項服務可用於在裝置位置變更或無法再判斷時接收通知。這會傳遞 `LocationResult`，您可以在其中取得 `Location`，以便儲存至資料庫。

現在您已大致瞭解要做什麼，可以開始編寫程式碼了！

4. 新增地點功能

本程式碼研究室著重於最常見的位置資訊選項：「僅在使用該應用程式時允許」。

若要接收位置資訊更新，應用程式必須顯示活動，或是在前景執行服務 (並顯示通知)。

權限

本程式碼研究室的目的是說明如何接收位置更新通知，而非如何要求位置存取權，因此我們已為您編寫權限相關程式碼。如果您已瞭解這項概念，可以略過。

以下是權限重點 (這部分不需要採取任何行動)：

1. 在 `AndroidManifest.xml` 中宣告您使用的權限。
2. 嘗試存取位置資訊前，請先檢查使用者是否已授予應用程式相關權限。如果應用程式尚未取得權限，請要求存取權。
3. 處理使用者的權限選擇。(您可以在 `MainActivity.kt` 中查看這段程式碼)。

在 `AndroidManifest.xml` 或 `MainActivity.kt` 中搜尋 `TODO: Step 1.0, Review Permissions`，即可查看所有為權限編寫的程式碼。

詳情請參閱「[權限總覽](#)」。

現在，請開始編寫一些位置資訊程式碼。

查看更新位置資訊所需的主要變數

在 `base` 模組中，搜尋 `TODO: Step 1.1, Review variables` 中的

`ForegroundOnlyLocationService.kt` 檔案。

這個步驟不需要採取任何行動。您只需要查看下列程式碼區塊和註解，即可瞭解用於接收位置資訊更新的主要類別和變數。

```
// TODO: Step 1.1, Review variables (no changes).
// FusedLocationProviderClient - Main class for receiving location updates.
private lateinit var fusedLocationProviderClient: FusedLocationProviderClient

// LocationRequest - Requirements for the location updates, i.e., how often you
// should receive updates, the priority, etc.
private lateinit var locationRequest: LocationRequest

// LocationCallback - Called when FusedLocationProviderClient has a new Location.
private lateinit var locationCallback: LocationCallback

// Used only for local storage of the last known location. Usually, this would be saved to
// your
// database, but because this is a simplified sample without a full database, we only need
// the
// last location to create a Notification if the user navigates away from the app.
private var currentLocation: Location? = null
```

檢查 FusedLocationProviderClient 初始化作業

在 `base` 模組中，搜尋 `ForegroundOnlyLocationService.kt` 檔案中的 `TODO: Step 1.2, Review the FusedLocationProviderClient`。程式碼應如下所示：

```
// TODO: Step 1.2, Review the FusedLocationProviderClient.
fusedLocationProviderClient = LocationServices.getFusedLocationProviderClient(this)
```

如先前留言所述，這是取得位置資訊更新的主要類別。系統已為您初始化變數，但請務必檢查程式碼，瞭解變數的初始化方式。稍後您會在此處新增程式碼，要求更新位置資訊。

初始化 LocationRequest

1. 在 `base` 模組中，搜尋 `ForegroundOnlyLocationService.kt` 檔案中的 `TODO: Step 1.3, Create a LocationRequest`。
2. 在註解後方新增以下程式碼。

`LocationRequest` 初始化程式碼會為要求新增所需的額外服務品質參數 (間隔、最長等待時間和優先順序)。

```
// TODO: Step 1.3, Create a LocationRequest.
locationRequest = LocationRequest.create().apply {
    // Sets the desired interval for active location updates. This interval is inexact. You
    // may not receive updates at all if no location sources are available, or you may
    // receive them less frequently than requested. You may also receive updates more
    // frequently than requested if other applications are requesting location at a more
    // frequent interval.
    //
    // IMPORTANT NOTE: Apps running on Android 8.0 and higher devices (regardless of
    // targetSdkVersion) may receive updates less frequently than this interval when the app
    // is no longer in the foreground.
    interval = TimeUnit.SECONDS.toMillis(60)

    // Sets the fastest rate for active location updates. This interval is exact, and your
    // application will never receive updates more frequently than this value.
    fastestInterval = TimeUnit.SECONDS.toMillis(30)

    // Sets the maximum time when batched location updates are delivered. Updates may be
    // delivered sooner than this interval.
    maxWaitTime = TimeUnit.MINUTES.toMillis(2)

    priority = LocationRequest.PRIORITY_HIGH_ACCURACY
}
```

3. 請詳閱註解，瞭解各項指令的作用。

初始化 LocationCallback

1. 在 `base` 模組中，搜尋 `ForegroundOnlyLocationService.kt` 檔案中的 `TODO: Step 1.4, Initialize the LocationCallback`。
2. 在註解後方新增以下程式碼。

```
// TODO: Step 1.4, Initialize the LocationCallback.
locationCallback = object : LocationCallback() {
    override fun onLocationResult(locationResult: LocationResult) {
        super.onLocationResult(locationResult)

        // Normally, you want to save a new location to a database. We are simplifying
        // things a bit and just saving it as a local variable, as we only need it again
        // if a Notification is created (when the user navigates away from app).
        currentLocation = locationResult.lastLocation

        // Notify our Activity that a new location was added. Again, if this was a
        // production app, the Activity would be listening for changes to a database
        // with new locations, but we are simplifying things a bit to focus on just
        // learning the location side of things.
        val intent = Intent(ACTION_FOREGROUND_ONLY_LOCATION_BROADCAST)
        intent.putExtra(EXTRA_LOCATION, currentLocation)
        LocalBroadcastManager.getInstance(applicationContext).sendBroadcast(intent)

        // Updates notification content if this service is running as a foreground
        // service.
        if (serviceRunningInForeground) {
            notificationManager.notify(
                NOTIFICATION_ID,
                generateNotification(currentLocation))
        }
    }
}
```

您在此建立的 `LocationCallback` 是 `FusedLocationProviderClient` 在有新的位置更新通知時會呼叫的回應。

在回應中，您首先會使用 `LocationResult` 物件取得最新位置資訊。之後，請使用本機廣播 (如已啟用) 通知 `Activity` 新位置，或更新 `Notification` (如果這項服務以前景 `Service` 執行)。

3. 請詳閱註解，瞭解各個部分的作用。

訂閱位置資訊變更

現在您已初始化所有項目，需要讓 `FusedLocationProviderClient` 知道您想接收更新。

1. 在 `base` 模組中，搜尋 `ForegroundOnlyLocationService.kt` 檔案中的 `Step 1.5, Subscribe to location changes`。
2. 在註解後方新增以下程式碼。

```
// TODO: Step 1.5, Subscribe to location changes.
fusedLocationProviderClient.requestLocationUpdates(locationRequest, locationCallback,
    Looper.getMainLooper())
```

`requestLocationUpdates()` 呼叫會通知 `FusedLocationProviderClient`，您想接收位置更新。

您應該會認出先前定義的 `LocationRequest` 和 `LocationCallback`。這些參數會讓 `FusedLocationProviderClient` 瞭解要求的服務品質參數，以及更新時應呼叫的內容。最後，`Looper` 物件會指定回呼的執行緒。

您可能也會注意到，這段程式碼位於 `try/catch` 陳述式中。如果應用程式沒有位置資訊存取權，就會發生 `SecurityException`，因此需要這個區塊。

取消訂閱位置資訊變更

如果應用程式不再需要存取位置資訊，請務必取消訂閱位置資訊更新。

1. 在 `base` 模組中，搜尋 `ForegroundOnlyLocationService.kt` 檔案中的 `TODO: Step 1.6, Unsubscribe to location changes`。
2. 在註解後方新增以下程式碼。

```
// TODO: Step 1.6, Unsubscribe to location changes.
val removeTask = fusedLocationProviderClient.removeLocationUpdates(locationCallback)
removeTask.addOnCompleteListener { task ->
    if (task.isSuccessful) {
        Log.d(TAG, "Location Callback removed.")
        stopSelf()
    } else {
        Log.d(TAG, "Failed to remove Location Callback.")
    }
}
```

`removeLocationUpdates()` 方法會設定工作，讓 `FusedLocationProviderClient` 知道您不再想接收 `LocationCallback` 的位置資訊更新。`addOnCompleteListener()` 會提供完成的回呼，並執行 `Task`。

如上一個步驟所述，您可能會發現這段程式碼位於 `try/catch` 陳述式中。如果應用程式沒有存取位置資訊的權限，就會發生 `SecurityException`，因此需要這個區塊。

您可能會想知道何時會呼叫含有訂閱/取消訂閱程式碼的方法。使用者輕觸按鈕時，系統會在主要類別中觸發這些事件。如要查看，請前往 `MainActivity.kt` 課程。

執行 app

從 Android Studio 執行應用程式，然後試用位置資訊按鈕。

輸出畫面應會顯示位置資訊。這是適用於 Android 9 的完整應用程式。

注意：如果您已接受權限，系統就不會顯示新的位置存取權對話方塊。如要查看，請前往應用程式資訊畫面，移除應用程式的位置資訊存取權。

9:00



Location in Android

START RECEIVING LOCATION UPDATES



Allow **Location in Android** to access this device's location?

☐ Don't ask again

DENY

ALLOW

9:00



Location in Android

STOP RECEIVING LOCATION UPDATES

Foreground location: (37.41188, -122.07832)



步驟 5 · 約 6 分鐘

5. 支援 Android 10

在本節中，您將新增 Android 10 的支援。

您的應用程式已訂閱位置資訊變更，因此不需要進行太多作業。

事實上，您只需要指定前景服務用於定位目的即可。

目標 SDK 29

1. 在 `base` 模組中，搜尋 `build.gradle` 檔案中的 `TODO: Step 2.1, Target Android 10 and then Android 11.`。
2. 進行下列變更：
3. 將 `targetSdkVersion` 設為 `29`。

程式碼應如下所示：

```
android {  
    // TODO: Step 2.1, Target Android 10 and then Android 11.  
    compileSdkVersion 29  
    defaultConfig {  
        applicationId "com.example.android.whileinuselocation"  
        minSdkVersion 26  
        targetSdkVersion 29  
        versionCode 1  
        versionName "1.0"  
    }  
    ...  
}
```

完成後，系統會要求您同步處理專案。按一下「立即同步」。

Gradle files have changed since last project sync. A project sync may be necessary for the IDE to work p... [Sync Now](#)

完成後，您的應用程式就差不多可以支援 Android 10 了。

新增前景服務類型

在 Android 10 中，如需「僅限使用期間」的位置存取權，請務必加入前景服務類型。在本例中，這項權限是用來取得位置資訊。

在 `base` 模組中，搜尋 `AndroidManifest.xml` 中的 `TODO: 2.2, Add foreground service type`，並將下列程式碼新增至 `<service>` 元素：

```
android:foregroundServiceType="location"
```

程式碼應如下所示：

```
<application>
    ...

    <!-- Foreground services in Android 10+ require type. -->
    <!-- TODO: 2.2, Add foreground service type. -->
    <service
        android:name="com.example.android.whileinuselocation.ForegroundOnlyLocationService"
        android:enabled="true"
        android:exported="false"
        android:foregroundServiceType="location" />
</application>
```

大功告成！您的應用程式遵循 Android 位置資訊最佳做法，支援 Android 10 的「僅限使用期間」位置資訊。

執行 app

從 Android Studio 執行應用程式，然後試用位置資訊按鈕。

一切都應照常運作，但現在適用於 Android 10。如果您先前未接受位置資訊權限，現在應該會看到權限畫面！

注意：如果您已接受權限，系統就不會顯示新的位置存取權對話方塊。如要查看，請前往應用程式資訊畫面，移除應用程式的位置存取權。

10:00



Location in Android

START RECEIVING LOCATION UPDATES



Allow **Location in Android** to
access this device's location?

Allow only while using the app

Deny

Deny & don't ask again



10:00



Location in Android

STOP RECEIVING LOCATION UPDATES

Foreground location: (37.26844, -121.99562)



3:51

Tue, Apr 7

LTE 100%



Location in Android • now

Location in Android
(37.3810033, -122.0680767)

Launch activity Stop receiving location updates

Silent notifications

Android System

USB debugging connected
Tap to turn off USB debugging

Android System • Charging this device via USB

Manage



6. 支援 Android 11

在本節中，您會將 Android 11 設為目標。

好消息！除了 `build.gradle` 檔案，您不需要變更任何檔案！

目標 SDK 11

1. 在 `base` 模組中，搜尋 `build.gradle` 檔案中的 **TODO: Step 2.1, Target SDK**。
2. 進行下列變更：
3. `compileSdkVersion` 至 30
4. `targetSdkVersion` 至 30

程式碼應如下所示：

```
android {  
    TODO: Step 2.1, Target Android 10 and then Android 11.  
    compileSdkVersion 30  
    defaultConfig {  
        applicationId "com.example.android.whileinuselocation"  
        minSdkVersion 26  
        targetSdkVersion 30  
        versionCode 1  
        versionName "1.0"  
    }  
    ...  
}
```

完成後，系統會要求您同步處理專案。按一下「立即同步」。

Gradle files have changed since last project sync. A project sync may be necessary for the IDE to work p... [Sync Now](#)

完成後，您的應用程式就能支援 Android 11 了！

執行 app

從 Android Studio 執行應用程式，然後嘗試點選按鈕。

一切應可照常運作，但現在適用於 Android 11。如果您先前未接受位置資訊權限，現在應該會看到權限畫面！

注意：如果您已接受權限，系統就不會顯示新的位置存取權對話方塊。如要查看，請前往應用程式資訊畫面，移除應用程式的位置存取權。

10:10



Location in Android

START RECEIVING LOCATION UPDATES



**Allow Location in Android to
access this device's location?**

While using the app

Only this time

Deny



10:10



Location in Android

STOP RECEIVING LOCATION UPDATES

Foreground location: (37.2682205, -121.9955512)



7. Android 適用的位置資訊策略

按照本程式碼研究室所示方式檢查及要求位置存取權，應用程式就能順利追蹤裝置位置資訊的存取層級。

本頁列出幾項與位置資訊存取權相關的重要最佳做法。如要進一步瞭解如何保護使用者資料安全，請參閱「[應用程式權限最佳做法](#)」。

只要求必要權限

僅在必要時要求權限。例如：

- 除非絕對必要，否則請勿在應用程式啟動時要求位置存取權。
- 如果您的應用程式指定 Android 10 以上版本，且有前景服務，請在資訊清單中聲明 `foregroundServiceType` 為 `"location"`。
- 除非有[這篇文章](#)所述的有效用途，否則請勿要求背景位置資訊存取權。

如果未授予權限，請支援優雅降級

為維持良好的使用者體驗，請設計應用程式，以便妥善處理下列情況：

- 您的應用程式無法存取任何位置資訊。
 - 應用程式在背景執行時無法存取位置資訊。
-

步驟 8 · 約 1 分鐘

8. 恭喜

您已學會在 Android 中接收位置資訊更新，並瞭解最佳做法！

瞭解詳情

- 如果您有正當用途，請參閱[完整範例](#)，瞭解如何使用背景位置資訊
 - [要求位置更新通知](#)
-